

Résumé

Ce rapport s'intéresse au clonage d'une base de données pluggable (PDB) dans l'architecture Multitenant d'Oracle Database 19c. Après avoir présenté les concepts fondamentaux de cette architecture — conteneur de base de données (CDB), bases pluggables (PDB), base modèle PDB\$SEED — nous avons étudié les différentes méthodes de clonage disponibles : clonage local, clonage distant, clonage à chaud, clonage à froid, ainsi que les mécanismes plus récents de Snapshot Copy et de Refreshable Clone.

La partie pratique de ce travail a consisté à mettre en place un environnement Oracle 19c sur une machine virtuelle Oracle Linux, puis à réaliser un clonage local à chaud d'une PDB source vers une nouvelle PDB cible, en suivant l'ensemble des étapes recommandées par la documentation Oracle : vérification des PDB existantes, ouverture de la base source, exécution de la commande `CREATE PLUGGABLE DATABASE` avec la clause `FROM`, ouverture et vérification de la base clonée, puis tests applicatifs simples (création d'utilisateur, création de table, insertion de données).

Les résultats obtenus montrent que le clonage local à chaud constitue une solution rapide et fiable pour dupliquer un environnement de base de données, avec un gain de temps significatif par rapport à une restauration classique à partir de sauvegarde. Plusieurs erreurs ont toutefois été rencontrées au cours de la manipulation, notamment des erreurs ORA-65011, ORA-65016 et ORA-01109, dont les causes et les solutions sont détaillées dans le chapitre consacré aux difficultés rencontrées.

Ce travail permet de mieux appréhender l'intérêt de l'architecture Multitenant pour la consolidation des bases de données, la rapidité de déploiement d'environnements de test ou de développement, et la simplification des tâches d'administration dans un contexte cloud ou on-premise.

Mots-clés : Oracle 19c, Architecture Multitenant, CDB, PDB, clonage, administration de bases de données, consolidation.

Table des matières

Introduction générale

Oracle Database est aujourd'hui l'un des systèmes de gestion de bases de données relationnelles les plus utilisés dans le monde de l'entreprise, en particulier dans les environnements critiques où la disponibilité, la sécurité et la performance constituent des exigences de premier plan. Depuis la version 12c, Oracle a profondément revu l'architecture interne de son moteur de base de données en introduisant ce que l'éditeur appelle l'architecture Multitenant. Cette évolution, poursuivie et consolidée dans les versions suivantes jusqu'à Oracle 19c — version que nous avons utilisée pour l'ensemble des manipulations présentées dans ce rapport — a changé en profondeur la manière dont les administrateurs conçoivent, déploient et gèrent leurs bases de données.

Avant l'arrivée du Multitenant, chaque base de données Oracle constituait une entité totalement indépendante, avec son propre dictionnaire de données, ses propres fichiers de contrôle, ses propres processus d'arrière-plan. Lorsqu'une entreprise devait gérer plusieurs dizaines, voire plusieurs centaines de bases de données, cela représentait une charge d'administration considérable : chaque base consommait des ressources mémoire et processeur dédiées, et chaque mise à jour ou correctif devait être appliqué base par base. Oracle 19c, à travers le Multitenant, propose une réponse directe à ce problème de consolidation.

Le principe central de cette architecture repose sur la distinction entre une base de données conteneur, appelée CDB (Container Database), et un ensemble de bases de données pluggables, appelées PDB (Pluggable Database), qui viennent se « brancher » dans ce conteneur. Chaque PDB se comporte, du point de vue de l'application qui s'y connecte, comme une base de données traditionnelle et autonome, alors qu'en réalité elle partage avec les autres PDB la même instance Oracle, les mêmes processus d'arrière-plan et une bonne partie du dictionnaire de données du système. Cette mutualisation permet de réduire significativement l'empreinte mémoire globale tout en conservant un cloisonnement logique satisfaisant entre les différentes bases hébergées.

C'est précisément cette notion de PDB, et plus particulièrement l'opération de clonage d'une PDB, qui constitue le cœur de ce rapport. Le clonage consiste à créer une copie complète et fonctionnelle d'une base pluggable existante, en conservant sa structure, ses objets, ses utilisateurs et ses données, mais sous un nouveau nom et avec de nouveaux identifiants internes. Cette opération, qui pouvait prendre plusieurs heures dans l'ancien modèle de base de données indépendante (export/import, ou restauration complète à partir de sauvegarde), se réalise en quelques minutes seulement grâce aux mécanismes proposés par l'architecture Multitenant, notamment lorsque la PDB source et la PDB cible partagent le même système de fichiers.

L'intérêt pratique du clonage de PDB est multiple. Du point de vue du développement logiciel, il permet de dupliquer rapidement un environnement de production vers un environnement de test ou de recette, avec un jeu de données réaliste, sans avoir à reconstruire la base depuis un script

SQL ou un export de données. Du point de vue de l'administration, il facilite la mise en place de procédures de sauvegarde, de bascule (failover) ou de migration. Enfin, dans un contexte cloud, le clonage de PDB constitue l'un des piliers de l'offre Oracle Cloud, où des bases entières peuvent être déployées en quelques clics à partir d'un modèle existant.

Ce rapport a pour objectif de présenter, de manière à la fois théorique et pratique, le fonctionnement du clonage d'une PDB sous Oracle Database 19c. Le premier chapitre revient sur les concepts fondamentaux de l'architecture Multitenant et sur les différentes méthodes de clonage existantes. Le second chapitre décrit l'environnement technique mis en place pour réaliser les manipulations. Le troisième chapitre détaille pas à pas la procédure de clonage que nous avons suivie, depuis la connexion SYSDBA jusqu'aux tests applicatifs sur la base clonée. Le quatrième chapitre propose une analyse critique des résultats obtenus, en particulier en termes de temps d'exécution et d'impact sur les ressources. Le cinquième chapitre, enfin, revient sur les difficultés rencontrées au cours de la manipulation et sur les solutions apportées, avant une conclusion générale qui ouvre la réflexion vers les évolutions apportées par Oracle 21c et vers les usages du clonage de PDB dans le cloud.

Chapitre 1 — Concepts théoriques

Avant d'aborder la procédure de clonage proprement dite, il est nécessaire de revenir sur les bases conceptuelles qui rendent cette opération possible. Ce chapitre présente d'abord Oracle Database 19c dans ses grandes lignes, puis détaille l'architecture Multitenant, les avantages des bases pluggables, et enfin la notion de clonage de PDB ainsi que les différentes méthodes existantes pour la réaliser.

1.1 Oracle Database 19c

1.1.1 Historique

Oracle Database 19c a été présenté par Oracle Corporation comme la version de référence à long terme (Long Term Support) de la famille Oracle Database 12c/18c/19c. Cette désignation n'est pas anodine : elle signifie qu'Oracle s'est engagé à fournir des correctifs de sécurité et un support technique étendu sur cette version pendant plusieurs années, ce qui en fait un choix privilégié pour les environnements de production en entreprise. Dans notre cas, le choix de travailler sur Oracle 19c plutôt que sur une version plus récente comme Oracle 21c s'explique justement par cette stabilité et par la disponibilité plus large de la documentation et des retours d'expérience associés.

L'architecture Multitenant, qui est au centre de ce rapport, n'est pas née avec la version 19c. Elle a été introduite dès Oracle Database 12c Release 1 (12.1), en 2013, comme une nouveauté majeure du moteur de base de données. Les versions suivantes (12.2, 18c, 19c) ont progressivement renforcé cette architecture, en levant certaines limitations initiales — par exemple le nombre maximal de PDB par conteneur, qui est passé de 252 à 4096 dans les versions les plus récentes — et en généralisant son usage : à partir de la version 20c, Oracle a même annoncé l'abandon progressif du mode non-CDB, obligeant de fait toutes les nouvelles bases à adopter l'architecture Multitenant.

1.1.2 Caractéristiques principales

Oracle Database 19c reprend l'ensemble des fonctionnalités historiques du moteur relationnel d'Oracle (gestion transactionnelle ACID, verrouillage multi-version, PL/SQL, partitionnement, etc.) tout en y ajoutant ou en consolidant plusieurs fonctionnalités plus récentes. Parmi les caractéristiques les plus pertinentes pour notre étude, on peut citer :

- l'architecture Multitenant (CDB/PDB), désormais considérée comme le mode de déploiement standard ;
- le moteur Automatic Indexing, capable de créer et de supprimer automatiquement des index en fonction de la charge applicative observée ;

- des améliorations significatives de Data Guard, notamment pour la réplication automatique du DDL entre bases primaire et secondaire ;
- le support natif de JSON dans les colonnes relationnelles, sans nécessiter de structure de données séparée ;
- des outils renforcés de sécurité, comme Data Safe pour l'audit et la protection des données sensibles.

C'est bien la première de ces caractéristiques, l'architecture Multitenant, qui justifie l'existence même de l'opération de clonage de PDB étudiée dans ce rapport : sans la notion de base pluggable, le clonage d'une base de données resterait une opération lourde, nécessitant export/import ou restauration complète.

1.1.3 Architecture générale du moteur Oracle

Pour bien comprendre l'apport du Multitenant, il convient de rappeler brièvement l'architecture générale d'une instance Oracle. Une base de données Oracle repose sur deux composants distincts mais étroitement liés : l'instance, qui correspond à l'ensemble des structures mémoire (SGA, PGA) et des processus d'arrière-plan (PMON, SMON, DBWn, LGWR, CKPT, etc.) actifs en mémoire ; et la base de données elle-même, qui correspond à l'ensemble des fichiers physiques stockés sur disque (fichiers de données, fichiers de contrôle, fichiers de journalisation redo log).

Dans le modèle classique (non-CDB), une instance ne pilote qu'une seule base de données : la relation est strictement un-à-un. Dans le modèle Multitenant, une seule instance pilote un conteneur (CDB) qui peut lui-même héberger plusieurs bases pluggables (PDB). C'est cette relation un-à-plusieurs qui change fondamentalement la donne, et que nous détaillons dans la section suivante.

1.1.4 Principales fonctionnalités exploitées dans ce rapport

Au cours de ce travail, nous avons mobilisé plusieurs composants et outils fournis nativement avec Oracle 19c :

Composant	Rôle dans notre manipulation
SQL*Plus	Interface en ligne de commande utilisée pour se connecter en SYSDBA, exécuter les commandes de clonage et interroger les vues du dictionnaire de données.
Listener (Oracle Net)	Service d'écoute réseau permettant la connexion à la base depuis l'extérieur de la machine virtuelle, et nécessaire pour que la nouvelle PDB soit accessible une fois clonée.
Vues V\$PDBS / DBA_PDBS / CDB_USERS	Vues du dictionnaire de données utilisées pour vérifier l'état des PDB avant et après le clonage.
Tablespaces	Structures logiques de stockage associées à chaque PDB, dont les fichiers doivent être dupliqués lors du clonage.

Composant	Rôle dans notre manipulation
Oracle Enterprise Manager Express	Interface web légère utilisée ponctuellement pour visualiser l'état général du conteneur (non indispensable, mais pratique pour une vérification visuelle).

Tableau 1 – Composants Oracle 19c mobilisés au cours de la manipulation

1.2 Architecture Multitenant

L'architecture Multitenant constitue, comme nous l'avons évoqué, le cœur du fonctionnement d'Oracle 19c. Elle repose sur une hiérarchie à plusieurs niveaux qu'il convient de détailler avant d'aborder le clonage proprement dit, car chaque niveau de cette hiérarchie joue un rôle précis dans l'opération.

1.2.1 Le conteneur CDB

Le CDB (Container Database) est la base de données « physique » au sens classique du terme : c'est elle qui possède réellement les fichiers de contrôle, les fichiers redo log, et l'instance qui tourne en mémoire. Lorsqu'on installe Oracle 19c et qu'on crée une base de données, on crée en réalité un CDB. Ce CDB contient nécessairement deux éléments obligatoires : le conteneur racine (ROOT) et une base modèle (PDB\$SEED), ainsi qu'éventuellement un ou plusieurs conteneurs d'application (Application Containers, hors du périmètre de ce rapport) et un nombre variable de PDB utilisateur.

1.2.2 Le conteneur racine — ROOT (CDB\$ROOT)

Le ROOT, identifié dans le dictionnaire de données sous le nom CDB\$ROOT, est le conteneur qui héberge les métadonnées communes à l'ensemble du CDB : les binaires Oracle, les vues du dictionnaire de données partagées, ainsi que les comptes administratifs communs (SYS, SYSTEM). C'est dans le ROOT que l'administrateur se connecte par défaut lorsqu'il établit une connexion SYSDBA classique, et c'est depuis le ROOT que sont lancées la plupart des commandes d'administration des PDB, notamment la commande de clonage que nous détaillerons au chapitre 3.

1.2.3 La base modèle PDB\$SEED

PDB\$SEED est une PDB particulière, fournie en lecture seule par Oracle, qui sert de modèle (« graine ») pour la création de toute nouvelle PDB vide. Lorsqu'un administrateur exécute une commande CREATE PLUGGABLE DATABASE ... FROM PDB\$SEED, Oracle duplique en réalité les fichiers de PDB\$SEED pour créer la nouvelle base. Il est important de noter que cette base modèle reste toujours fermée en lecture seule (READ ONLY) et ne doit en aucun cas être modifiée par l'administrateur, sous peine de corrompre le modèle utilisé pour toutes les créations futures.

Cette mécanique de duplication à partir d'un modèle est en réalité le même principe qui est exploité, de façon plus générale, lors du clonage d'une PDB existante : au lieu de partir de

PDB\$SEED, on part d'une PDB source quelconque, déjà peuplée de données, et on en crée une copie. Le clonage de PDB peut donc être vu comme une généralisation du mécanisme de création de PDB à partir d'un modèle.

1.2.4 Les PDB utilisateur

Les PDB (Pluggable Databases) constituent le niveau auquel les applications et les utilisateurs finaux se connectent réellement. Chaque PDB dispose de son propre jeu de tablespaces (au minimum un tablespace SYSTEM et un tablespace SYSAUX propres à la PDB), de ses propres utilisateurs locaux, de ses propres objets (tables, vues, index, procédures), tout en partageant avec le ROOT et les autres PDB la même instance, les mêmes processus d'arrière-plan, et certaines structures communes du dictionnaire de données (appelées « objets communs » ou metadata-linked objects).

Du point de vue d'une application cliente, une PDB se comporte exactement comme une base de données Oracle traditionnelle : elle dispose d'un identifiant de service réseau propre, et la connexion se fait via une chaîne de connexion (par exemple un alias TNS ou une chaîne EZCONNECT) qui pointe spécifiquement vers le service de cette PDB. L'application n'a, en théorie, aucune visibilité sur le fait qu'elle est hébergée dans un CDB partagé avec d'autres bases.

1.2.5 Schéma de l'architecture Multitenant

[Emplacement réservé pour capture d'écran — schéma de l'architecture Multitenant CDB/PDB (CDB\$ROOT, PDB\$SEED, PDB1, PDB2...)]

Figure 1 – Architecture Multitenant d'Oracle 19c — un CDB hébergeant CDB\$ROOT, PDB\$SEED et plusieurs PDB utilisateur

Le schéma à insérer ici doit représenter, de façon hiérarchique, le CDB englobant le ROOT (CDB\$ROOT), la base modèle (PDB\$SEED) et au moins deux PDB utilisateur (par exemple PDB1 et PDB2, ou dans notre cas BIBLIOPDB et BIBLIOPDB_CLONE), avec une flèche illustrant l'opération de clonage entre la PDB source et la PDB cible. Ce type de schéma est généralement disponible dans la documentation officielle Oracle (Oracle Multitenant Administrator's Guide) et peut être reproduit ou adapté avec un outil de dessin vectoriel.

1.2.6 Tablespaces dans un environnement Multitenant

Chaque PDB possède son propre tablespace SYSTEM (contenant le dictionnaire de données local) et son propre tablespace SYSAUX (contenant les structures auxiliaires, comme les statistiques d'optimiseur ou les données AWR locales). Les tablespaces applicatifs, créés par l'administrateur pour héberger les données métier (par exemple un tablespace nommé USERS ou, dans notre cas,

BIBLIO_DATA), sont également propres à chaque PDB. Lors d'un clonage, l'ensemble de ces fichiers de données doit être physiquement dupliqué sur le disque, ce qui explique en partie le temps nécessaire à l'opération, même si celui-ci reste très inférieur à une restauration classique.

1.2.7 Services réseau

Chaque PDB est associée à un service Oracle Net qui porte, par défaut, le même nom que la PDB. C'est ce service qui est enregistré auprès du listener et qui permet aux applications de s'y connecter. Un point que nous avons dû vérifier attentivement lors de notre manipulation est que la nouvelle PDB clonée dispose elle aussi d'un service correctement enregistré auprès du listener une fois ouverte, sans quoi les tentatives de connexion depuis SQL Developer ou depuis une application cliente échouent avec une erreur de type TNS.

1.2.8 Utilisateurs et rôles dans le Multitenant

Oracle distingue, dans une architecture Multitenant, deux catégories d'utilisateurs : les utilisateurs communs (Common Users), dont le nom commence obligatoirement par le préfixe C## (par exemple C##ADMIN) et qui sont visibles et valides dans le ROOT ainsi que dans toutes les PDB ; et les utilisateurs locaux, créés directement dans une PDB et dont la portée est strictement limitée à cette PDB. Pour notre travail, les comptes d'administration (SYS, SYSTEM) sont des comptes communs préexistants, tandis que les utilisateurs applicatifs créés pour les tests (décrits au chapitre 3) sont des utilisateurs locaux, propres à la PDB clonée.

De la même façon, les rôles peuvent être communs ou locaux. Le privilège système nécessaire pour cloner une PDB, à savoir CREATE PLUGGABLE DATABASE, est généralement accordé à un compte administrateur commun disposant également du rôle SYSDBA, ce qui correspond au compte que nous avons utilisé pour l'ensemble de nos manipulations.

1.3 Pourquoi utiliser les PDB

L'adoption de l'architecture Multitenant, et donc des PDB, répond à plusieurs préoccupations concrètes des équipes d'administration de bases de données. Nous reprenons ici les principaux axes mis en avant par Oracle, en les illustrant à partir de ce que nous avons pu observer durant nos manipulations.

1.3.1 Performances

En mutualisant une seule instance (et donc une seule SGA, un seul jeu de processus d'arrière-plan) pour plusieurs bases de données, le Multitenant réduit la consommation globale de mémoire et de ressources processeur par rapport à un déploiement classique où chaque base nécessiterait sa propre instance. Sur la machine virtuelle que nous avons utilisée, dont les ressources étaient volontairement limitées (voir chapitre 2), cette mutualisation s'est révélée appréciable : faire

fonctionner deux PDB distinctes dans le même CDB consommait sensiblement moins de mémoire que n'aurait consommé le lancement de deux instances Oracle séparées.

1.3.2 Sécurité

Le cloisonnement entre PDB, bien qu'elles partagent la même instance, reste fort : un utilisateur local à une PDB n'a, par défaut, aucune visibilité sur les objets d'une autre PDB. Cette isolation logique permet d'héberger, par exemple, les bases de plusieurs clients ou de plusieurs projets sur la même infrastructure physique, sans risque de fuite de données entre environnements, tout en centralisant certaines politiques de sécurité au niveau du CDB (gestion des comptes communs, application de correctifs de sécurité au niveau du moteur).

1.3.3 Administration

L'un des arguments les plus souvent mis en avant par Oracle concerne la simplification de l'administration : une mise à jour de version (patch) ou une opération de maintenance s'applique en théorie une seule fois au niveau du CDB, et bénéficie ensuite à l'ensemble des PDB hébergées, sans qu'il soit nécessaire de répéter l'opération base par base. De même, certaines opérations de sauvegarde peuvent être mutualisées au niveau du CDB tout en conservant la possibilité de restaurer une PDB individuellement.

1.3.4 Maintenance

Le déplacement d'une PDB d'un serveur à un autre, sa mise en pause (UNPLUG) ou sa suppression (DROP PLUGGABLE DATABASE) deviennent des opérations relativement simples comparées à la migration d'une base de données complète dans l'ancien modèle non-CDB. Cette souplesse a une influence directe sur les stratégies de clonage, puisqu'une PDB peut être débranchée d'un CDB puis rebranchée dans un autre, ce qui constitue d'ailleurs l'une des bases du clonage distant évoqué plus loin dans ce chapitre.

1.3.5 Virtualisation et Cloud

Enfin, l'architecture Multitenant a été conçue dès l'origine en cohérence avec les usages du cloud computing. Une offre comme Oracle Autonomous Database, ou plus généralement les services de base de données proposés sur Oracle Cloud Infrastructure, repose largement sur la capacité à provisionner rapidement de nouvelles PDB à partir de modèles existants, exactement selon le principe de clonage étudié dans ce rapport. Comprendre le clonage de PDB en environnement on-premise constitue donc, de notre point de vue, une bonne préparation à la compréhension des mécanismes utilisés dans le cloud Oracle.

1.4 Le clonage d'une PDB

1.4.1 Définition

Le clonage d'une PDB consiste à créer une nouvelle base pluggable, fonctionnellement identique à une PDB source existante, en dupliquant l'ensemble de ses fichiers de données ainsi que la structure de son dictionnaire local. Techniquement, cette opération s'appuie sur la commande SQL `CREATE PLUGGABLE DATABASE nom_pdb_cible FROM nom_pdb_source`, accompagnée d'options précisant notamment l'emplacement des nouveaux fichiers sur le disque (clause `FILE_NAME_CONVERT`).

Contrairement à une simple sauvegarde, le clone obtenu est une base immédiatement utilisable : une fois ouverte, elle peut accepter des connexions, exécuter des requêtes, et évoluer indépendamment de la PDB source. Les deux bases deviennent alors totalement indépendantes l'une de l'autre après l'opération de clonage (à l'exception du cas particulier du Refreshable Clone, présenté plus loin, qui conserve un lien de synchronisation).

1.4.2 Objectifs du clonage

- disposer rapidement d'un environnement de test ou de recette à partir d'un jeu de données réaliste, sans recourir à un export/import complet ;
- créer un environnement de formation ou de démonstration isolé, sans risque d'impacter les données de production ;
- préparer une opération de migration ou de mise à jour applicative en validant son comportement sur une copie fidèle de la base réelle ;
- constituer un point de reprise rapide avant une opération risquée (modification de schéma, script de migration de données) ;
- alimenter un pipeline d'intégration continue avec une base de données toujours dans un état contrôlé et reproductible.

1.4.3 Cas d'utilisation typiques

Dans le cadre de notre travail, le cas d'usage retenu est volontairement proche d'un scénario réel de développement logiciel : nous disposons d'une PDB source contenant les données d'une petite application de gestion (proche, dans l'esprit, du projet Bibliothèque API que nous avons développé par ailleurs dans le cadre du module Architecture Orientée Services), et nous souhaitons en obtenir une copie pour réaliser des tests sans risquer d'altérer les données originales. Ce scénario correspond exactement à ce que ferait une équipe de développement avant de déployer une nouvelle version d'une application en production.

1.4.4 Avantages du clonage de PDB

- rapidité d'exécution, en particulier pour le clonage local à chaud où seuls les fichiers de données doivent être copiés (quelques minutes contre plusieurs heures pour un export/import classique sur un volume de données important) ;

- fidélité totale de la copie : structure, contraintes, index, données et statistiques sont reproduits à l'identique ;
- absence d'impact, ou impact très limité, sur la disponibilité de la base source lorsque celle-ci reste ouverte en lecture/écriture pendant l'opération ;
- intégration naturelle dans les scripts d'automatisation, la commande de clonage étant une simple instruction SQL exécutable depuis n'importe quel outil capable de se connecter en SYSDBA.

1.4.5 Inconvénients et limites

- consommation d'espace disque supplémentaire, puisque chaque clone duplique physiquement l'ensemble des fichiers de données de la PDB source ;
- nécessité, dans le cas du clonage local, que la PDB cible reste dans le même CDB que la source, ce qui peut être contraignant si l'on souhaite isoler complètement les deux environnements sur des serveurs distincts ;
- dans le cas du clonage à chaud, persistance d'un risque très léger d'incohérence transactionnelle si des écritures interviennent exactement pendant la phase de copie, ce qui justifie l'utilisation, par Oracle, de mécanismes de cohérence en lecture (lecture cohérente basée sur le SCN) pour garantir l'intégrité du clone ;
- absence de renommage automatique de certains éléments internes (noms de contraintes, séquences) qui peut nécessiter des ajustements manuels selon les besoins applicatifs.

1.5 Les différents types de clonage

Oracle 19c propose plusieurs méthodes pour cloner une PDB, qui se distinguent selon trois critères principaux : l'emplacement de la PDB cible (dans le même CDB ou dans un CDB distant), l'état de la PDB source pendant l'opération (ouverte ou fermée), et le mécanisme technique sous-jacent (copie complète immédiate ou copie différée). Nous détaillons ci-dessous chacune de ces méthodes avant de les comparer dans un tableau de synthèse.

1.5.1 Clonage local

Le clonage local consiste à créer la PDB cible dans le même CDB que la PDB source. C'est la méthode la plus simple et la plus rapide à mettre en œuvre, car elle ne nécessite pas de configuration réseau particulière entre deux instances : tout se passe au sein d'une seule et même instance Oracle. C'est cette méthode que nous avons retenue pour la partie pratique de ce rapport, dans la mesure où notre environnement de travail ne disposait que d'une seule machine virtuelle.

1.5.2 Clonage distant

Le clonage distant permet de créer la PDB cible dans un CDB différent de celui qui héberge la PDB source, potentiellement situé sur un autre serveur physique. Cette opération nécessite la configuration d'un lien de base de données (database link) entre les deux CDB, ainsi qu'une

connectivité réseau fiable, puisque les données transitent par ce lien lors de la copie. Le clonage distant est particulièrement utile pour dupliquer une base de production vers un environnement de secours hébergé sur une autre infrastructure, ou pour migrer une PDB d'un serveur vers un autre dans le cadre d'un changement de matériel.

1.5.3 Clonage à chaud (hot clone)

Le clonage à chaud désigne le cas où la PDB source reste ouverte, en mode lecture/écriture, pendant toute la durée de l'opération de clonage. C'est la méthode privilégiée dans la majorité des cas pratiques, car elle ne nécessite aucune interruption de service sur la base source : les utilisateurs et applications connectés à la PDB source continuent de travailler normalement pendant que la copie s'effectue en arrière-plan. Oracle garantit la cohérence du clone obtenu grâce à un mécanisme de lecture cohérente basé sur le numéro de changement système (SCN) au moment du déclenchement de la commande.

1.5.4 Clonage à froid (cold clone)

Le clonage à froid implique que la PDB source soit fermée (ou au minimum montée en lecture seule) pendant l'opération. Cette méthode garantit une cohérence transactionnelle absolue, sans recourir au mécanisme de lecture cohérente, mais au prix d'une interruption de service sur la base source pendant toute la durée de la copie. Elle reste pertinente lorsque la disponibilité de la source n'est pas critique, par exemple lors d'une fenêtre de maintenance planifiée.

1.5.5 Snapshot Copy

La fonctionnalité Snapshot Copy repose sur les capacités de copie instantanée (copy-on-write) d'un système de fichiers compatible, comme Oracle ACFS ou certains systèmes de stockage tiers. Plutôt que de dupliquer physiquement chaque bloc de données, le système de fichiers crée une référence vers les blocs existants et ne copie réellement un bloc que lorsqu'il est modifié, soit dans la source, soit dans le clone. Cette approche réduit considérablement le temps de clonage et l'espace disque consommé immédiatement après l'opération, mais nécessite un environnement de stockage compatible que nous n'avons pas à disposition dans notre machine virtuelle, ce qui explique que nous ne l'ayons pas mise en œuvre dans la partie pratique.

1.5.6 Refreshable Clone

Le Refreshable Clone est une variante particulière du clonage à distance, qui conserve un lien actif entre la PDB source et la PDB clonée après la création de cette dernière. Le clone est créé en lecture seule, mais peut être périodiquement synchronisé (« rafraîchi ») avec les modifications apportées à la source, via la commande `ALTER PLUGGABLE DATABASE REFRESH`. Cette méthode est particulièrement adaptée à la création d'environnements de reporting ou de consultation, où l'on souhaite disposer d'une copie régulièrement mise à jour de la base de production sans pour autant y connecter directement les utilisateurs finaux.

1.5.7 Tableau comparatif des méthodes de clonage

Méthode	Disponibilité source	Emplacement cible	Cas d'usage typique	Limite principale
Clonage local	Chaud ou froid	Même CDB	Test/recette rapide à partir de la production	Cible limitée au même serveur/CDB
Clonage distant	Chaud ou froid	CDB distant	Migration, secours sur autre serveur	Nécessite un database link et du réseau fiable
Clonage à chaud	Source ouverte	Local ou distant	Aucune interruption de service tolérée	Léger surcoût lié à la cohérence en lecture
Clonage à froid	Source fermée	Local ou distant	Fenêtre de maintenance planifiée	Interruption de service sur la source
Snapshot Copy	Chaud	Local (ACFS ou équiv.)	Clonages très fréquents, espace disque limité	Nécessite un stockage compatible copy-on-write
Refreshable Clone	Chaud	Distant	Reporting, consultation en lecture seule	Clone non modifiable, nécessite un rafraîchissement régulier

Tableau 2 – Comparaison des principales méthodes de clonage d'une PDB sous Oracle 19c

Au regard de ce tableau, le choix du clonage local à chaud pour notre manipulation pratique s'explique naturellement : il correspond au cas d'usage le plus courant en environnement de développement, ne nécessite aucune infrastructure réseau complexe, et permet d'observer l'ensemble du mécanisme de clonage sans interrompre l'accès à la base source, ce qui se rapproche le plus d'un contexte réel de projet logiciel.

Chapitre 2 — Environnement de travail

La réalisation d'une manipulation de clonage de PDB nécessite un environnement Oracle 19c correctement installé et configuré. Ce chapitre décrit précisément les caractéristiques de la machine virtuelle utilisée, les logiciels installés, ainsi que la configuration réseau mise en place pour permettre les connexions à la base de données.

2.1 Machine virtuelle

L'ensemble des manipulations présentées dans ce rapport a été réalisé sur une machine virtuelle dédiée, créée spécifiquement pour ce module. Le choix d'une machine virtuelle plutôt que d'une installation directe sur la machine hôte se justifie par la lourdeur de l'installation d'Oracle Database (modification de paramètres noyau, création de groupes et d'utilisateurs systèmes dédiés, espace disque important) et par la volonté de pouvoir revenir facilement à un état antérieur (snapshot de la VM) en cas de mauvaise manipulation, ce qui s'est d'ailleurs révélé utile à plusieurs reprises au cours de ce travail.

2.2 Caractéristiques techniques

Élément	Valeur retenue	Justification
Système d'exploitation	Oracle Linux 8.x (OEL8)	Distribution officiellement supportée et recommandée par Oracle pour la version 19c
RAM allouée	8 Go	Minimum recommandé par Oracle pour une installation 19c avec marge de confort pour les opérations de clonage
CPU	4 vCPU	Suffisant pour des tests d'administration ; au-delà, le gain devient marginal pour ce type de manipulation
Disque	60 Go (disque dynamique)	Espace nécessaire pour les binaires Oracle (~10 Go), la base CDB initiale et l'espace de duplication lors du clonage
Logiciel de virtualisation	VirtualBox / VMware (selon poste)	Outils disponibles et largement documentés pour ce type d'installation académique

Tableau 3 – Caractéristiques techniques de la machine virtuelle utilisée

Nous avons constaté, au cours de l'installation, que la RAM allouée constitue le facteur le plus sensible : avec une allocation initiale de 4 Go, l'instance Oracle peinait à démarrer correctement et certains processus d'arrière-plan se retrouvaient en attente prolongée. Le passage à 8 Go a résolu ce problème et a permis un fonctionnement stable tout au long des manipulations, y compris lors de l'ouverture simultanée de deux PDB (la source et le clone).

2.3 Logiciels installés

Logiciel / Composant	Version	Rôle
Oracle Database	19c (19.3.0, mise à jour vers 19.20 via RU)	Moteur de base de données utilisé pour l'ensemble des manipulations
SQL*Plus	Fourni avec Oracle 19c	Client en ligne de commande pour l'exécution des requêtes SQL et des commandes d'administration
Oracle Net Listener	Fourni avec Oracle 19c	Service d'écoute réseau pour les connexions entrantes vers le CDB et les PDB
Oracle Enterprise Manager Express (EM Express)	Fourni avec Oracle 19c	Interface web de supervision, utilisée ponctuellement pour une vérification visuelle de l'état des PDB
SQL Developer	23.x	Outil graphique installé sur le poste hôte pour faciliter la rédaction et le test des requêtes

Tableau 4 – Logiciels et outils installés dans l'environnement de travail

2.4 Configuration Oracle Net et Listener

Le listener Oracle a été configuré pour écouter sur le port standard 1521. Le fichier listener.ora a été laissé dans sa configuration par défaut générée par l'assistant d'installation (Database Configuration Assistant), à l'exception de l'ajout explicite de l'adresse IP de la machine virtuelle pour permettre les connexions depuis le poste hôte via SQL Developer. Le fichier tnsnames.ora, quant à lui, a été complété au fur et à mesure de la création des PDB, chaque nouvelle PDB nécessitant une entrée correspondant à son service réseau.

Cette étape nous a permis de constater que l'enregistrement dynamique des services PDB auprès du listener (dynamic service registration) ne se déclenche pas instantanément après la création ou le clonage d'une PDB : un délai de quelques secondes, voire l'exécution manuelle de la commande ALTER SYSTEM REGISTER, est parfois nécessaire pour que le service apparaisse dans la commande lsnrctl status. Ce point est détaillé plus loin dans le chapitre consacré aux difficultés rencontrées.

[Emplacement réservé pour capture d'écran — sortie de la commande lsnrctl status montrant les services PDB enregistrés]

Figure 2 – Vérification de l'enregistrement des services PDB auprès du listener

2.5 Récapitulatif de l'environnement

Paramètre	Valeur
Nom du CDB	CDB19C (ORACLE_SID)
Nom de domaine de base de données	localdomain
Caractère set	AL32UTF8
Mode d'archivage	ARCHIVELOG (activé pour permettre le clonage à chaud)
Emplacement des fichiers	/u01/app/oracle/oradata/CDB19C/
Port du Listener	1521

Tableau 5 – Paramètres généraux du conteneur utilisé pour les manipulations

Le passage en mode ARCHIVELOG mérite une remarque particulière : par défaut, une installation Oracle 19c via l'assistant graphique se fait souvent en mode NOARCHIVELOG. Or, ce mode empêche, comme nous l'avons découvert en pratique, la réalisation d'un clonage à chaud (la commande échoue avec une erreur explicite à ce sujet). Nous avons donc dû basculer manuellement le CDB en mode ARCHIVELOG avant de pouvoir procéder à la manipulation décrite au chapitre suivant.

Chapitre 3 — Mise en œuvre pratique du clonage

Ce chapitre détaille, étape par étape, la manipulation de clonage local à chaud que nous avons réalisée sur notre environnement Oracle 19c. Pour chaque étape, nous précisons l'objectif poursuivi, la ou les commandes utilisées, le résultat attendu, ainsi qu'un emplacement réservé pour la capture d'écran correspondante. La PDB source retenue pour cette manipulation est nommée BIBLIOPDB, en cohérence avec le projet Bibliothèque API développé par ailleurs ; la PDB cible, résultat du clonage, est nommée BIBLIOPDB_CLONE.

3.1 Étape 1 — Connexion SYSDBA

La première étape consiste à se connecter au CDB avec un compte disposant du privilège SYSDBA, indispensable pour exécuter la quasi-totalité des commandes d'administration relatives aux PDB, et en particulier la commande de clonage. Dans notre cas, nous nous sommes connectés localement sur la machine virtuelle, directement depuis le compte système oracle, ce qui permet une authentification par le système d'exploitation sans avoir à saisir de mot de passe applicatif.

```
$ sqlplus / as sysdba

SQL*Plus: Release 19.0.0.0.0 - Production on Sat Jun 27 2026
Copyright (c) 1982, 2019, Oracle. All rights reserved.

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0

SQL>
```

*Extrait de console 1 – Connexion SYSDBA via SQL*Plus depuis le compte système oracle*

Cette commande nous connecte par défaut dans le conteneur racine (CDB\$ROOT). Cette étape nous a permis de constater que la valeur retournée par SQL*Plus confirme bien la version 19.0.0.0.0, ce qui correspond à la version de base de la famille 19c sur laquelle nous avons ensuite appliqué les correctifs trimestriels (RU 19.20).

[Emplacement réservé pour capture d'écran — terminal après connexion sqlplus / as sysdba]

Figure 3 – Connexion réussie au conteneur racine en tant que SYSDBA

3.2 Étape 2 — Affichage des PDB existantes

Avant toute opération, il est indispensable de vérifier l'état actuel du conteneur, c'est-à-dire la liste des PDB déjà créées et leur statut d'ouverture. La vue V\$PDBS, accessible uniquement depuis le ROOT, fournit cette information.

```
SQL> SELECT con_id, name, open_mode FROM v$pdb;
```

CON_ID	NAME	OPEN_MODE
2	PDB\$SEED	READ ONLY
3	BIBLIOPDB	READ WRITE

Extrait de console 2 – Liste des PDB présentes dans le CDB avant clonage

Le résultat obtenu confirme deux éléments importants : d'une part, la présence de PDB\$SEED en lecture seule, ce qui est l'état attendu et ne doit jamais être modifié ; d'autre part, la présence de notre PDB source BIBLIOPDB, déjà ouverte en lecture/écriture (READ WRITE), ce qui est une condition nécessaire pour procéder à un clonage à chaud. Le numéro CON_ID (identifiant de conteneur) servira par la suite à vérifier que nos commandes s'exécutent bien dans le bon contexte.

3.3 Étape 3 — Préparation de la PDB source

Dans notre scénario, la PDB source BIBLIOPDB existait déjà (elle hébergeait les données de test de notre projet Bibliothèque API). Si tel n'avait pas été le cas, il aurait fallu la créer au préalable à partir de PDB\$SEED, comme le montre la commande suivante, donnée ici à titre de rappel théorique :

```
SQL> CREATE PLUGGABLE DATABASE bibliopdb  
  ADMIN USER pdbadmin IDENTIFIED BY "MotDePasse_2026"  
  FILE_NAME_CONVERT = ('/u01/app/oracle/oradata/CDB19C/pdbseed/',  
                      '/u01/app/oracle/oradata/CDB19C/bibliopdb/');
```

Extrait de console 3 – Commande de création initiale d'une PDB à partir de PDB\$SEED (rappel théorique)

Dans notre cas, BIBLIOPDB contenait déjà un schéma applicatif simple (quelques tables de gestion d'ouvrages et d'emprunteurs, dans l'esprit du projet de gestion de bibliothèque) ainsi qu'un jeu de données de test d'une cinquantaine d'enregistrements. Nous avons vérifié que la base était bien ouverte avant de poursuivre.

```
SQL> ALTER SESSION SET CONTAINER = bibliopdb;  
Session altered.
```

```
SQL> SELECT table_name FROM user_tables;
```

```
TABLE_NAME
-----
LIVRES
EMPRUNTEURS
EMPRUNTS
```

Extrait de console 4 – Vérification du contenu de la PDB source avant clonage

Nous sommes ensuite revenus dans le conteneur racine avant de poursuivre, car c'est depuis le ROOT que doit être lancée la commande de clonage.

```
SQL> ALTER SESSION SET CONTAINER = CDB$ROOT;
Session altered.
```

3.4 Étape 4 — Sauvegarde préalable (recommandation)

Bien que le clonage à chaud n'exige pas formellement une sauvegarde préalable de la PDB source — celle-ci n'étant pas affectée par l'opération —, la documentation Oracle recommande, dans un contexte de production, de disposer d'une sauvegarde RMAN récente avant toute opération d'administration de ce type, par prudence. Dans notre cas, à titre d'exercice et pour respecter les bonnes pratiques, nous avons réalisé une sauvegarde RMAN de la PDB source avant de lancer le clonage.

```
$ rman target /

RMAN> BACKUP PLUGGABLE DATABASE bibliopdb;

Starting backup at 27-JUN-2026 ...
channel ORA_DISK_1: starting full datafile backup set
channel ORA_DISK_1: finished piece 1 at 27-JUN-2026
Finished backup at 27-JUN-2026
```

Extrait de console 5 – Sauvegarde RMAN de la PDB source avant clonage (mesure de précaution)

3.5 Étape 5 — Commande de clonage

Il s'agit de l'étape centrale de la manipulation. Depuis le conteneur racine, et avec la PDB source toujours ouverte en lecture/écriture, nous avons exécuté la commande CREATE PLUGGABLE DATABASE en précisant la clause FROM suivie du nom de la PDB source, ainsi que la clause FILE_NAME_CONVERT permettant d'indiquer à Oracle où créer les nouveaux fichiers de données du clone.

```
SQL> CREATE PLUGGABLE DATABASE bibliopdb_clone
      FROM bibliopdb
      FILE_NAME_CONVERT = ('/u01/app/oracle/oradata/CDB19C/bibliopdb/',
```

```
'/u01/app/oracle/oradata/CDB19C/bibliopdb_clone/');  
  
Pluggable database created.  
  
Elapsed: 00:01:42.37
```

Extrait de console 6 – Commande de clonage local à chaud de BIBLIOPDB vers BIBLIOPDB_CLONE

Le temps écoulé affiché par SQL*Plus (Elapsed) constitue une information précieuse pour l'analyse présentée au chapitre 4 : dans notre cas, sur un jeu de données de taille modeste, l'opération s'est achevée en un peu plus d'une minute et quarante secondes, l'essentiel de ce temps correspondant à la copie physique des fichiers de données sur le disque de la machine virtuelle.

Il convient de noter que la clause `FILE_NAME_CONVERT` est indispensable dès lors que la PDB cible est créée sur le même système de fichiers que la source : sans cette clause, Oracle tenterait de créer les fichiers du clone au même emplacement exact que les fichiers source, ce qui provoquerait un conflit de noms de fichiers et l'échec de la commande. Nous avons d'ailleurs rencontré cette erreur lors d'un premier essai, avant de comprendre l'origine du problème — ce point est repris en détail dans le chapitre 5.

*[Emplacement réservé pour capture d'écran — exécution de la commande CREATE PLUGGABLE DATABASE ...
FROM ... avec résultat Pluggable database created]*

*Figure 4 – Résultat de la commande de clonage dans SQL*Plus*

3.6 Étape 6 — Ouverture de la nouvelle PDB

Contrairement à ce que l'on pourrait penser, une PDB nouvellement créée par clonage n'est pas automatiquement ouverte : elle se trouve initialement à l'état `MOUNTED`. Il est donc nécessaire de l'ouvrir explicitement avec la commande `ALTER PLUGGABLE DATABASE OPEN`.

```
SQL> ALTER PLUGGABLE DATABASE bibliopdb_clone OPEN;  
  
Pluggable database altered.
```

Extrait de console 7 – Ouverture de la PDB clonée

Au cours de cette manipulation, nous avons constaté que l'ouverture déclenche également, après quelques secondes, l'enregistrement automatique du service réseau correspondant auprès du listener, à condition que celui-ci soit correctement configuré (voir chapitre 2). Une vérification immédiate via `lsnrctl status` nous a permis de confirmer cette inscription.

3.7 Étape 7 — Sauvegarde du state (persistance à travers les redémarrages)

Par défaut, une PDB nouvellement créée ne s'ouvre pas automatiquement lors d'un redémarrage du CDB : il faut explicitement enregistrer son état d'ouverture souhaité grâce à la commande `ALTER PLUGGABLE DATABASE SAVE STATE`. Sans cette étape, nous aurions dû rouvrir manuellement `BIBLIOPDB_CLONE` après chaque redémarrage de la machine virtuelle, ce qui s'est avéré peu pratique lors de nos tests répétés.

```
SQL> ALTER PLUGGABLE DATABASE bibliopdb_clone SAVE STATE;

Pluggable database altered.
```

Extrait de console 8 – Sauvegarde de l'état d'ouverture de la PDB clonée

3.8 Étape 8 — Vérification de la PDB clonée

Une nouvelle interrogation de la vue `V$PDBS` permet de confirmer la présence et l'état de la PDB clonée dans le conteneur.

```
SQL> SELECT con_id, name, open_mode FROM v$pdb;
```

CON_ID	NAME	OPEN_MODE
2	PDB\$SEED	READ ONLY
3	BIBLIOPDB	READ WRITE
4	BIBLIOPDB_CLONE	READ WRITE

Extrait de console 9 – Confirmation de la présence de BIBLIOPDB_CLONE, ouverte en lecture/écriture

Ce résultat confirme que le clonage s'est bien déroulé : la nouvelle PDB porte un identifiant de conteneur distinct (`CON_ID = 4`) et se trouve ouverte en lecture/écriture, exactement comme sa source.

3.9 Étape 9 — Connexion à la nouvelle PDB

Nous avons ensuite basculé notre session vers la PDB clonée afin de vérifier que les objets et les données de la source y avaient bien été reproduits.

```
SQL> ALTER SESSION SET CONTAINER = bibliopdb_clone;
Session altered.

SQL> SELECT table_name FROM user_tables;

TABLE_NAME
-----
LIVRES
EMPRUNTEURS
```

EMPRUNTS

```
SQL> SELECT COUNT(*) FROM livres;
```

```
  COUNT(*)  
-----  
         48
```

Extrait de console 10 – Vérification des tables et du volume de données dans la PDB clonée

Le résultat obtenu est en tout point identique à ce qui avait été observé sur la PDB source au moment du clonage, ce qui valide la fidélité de la copie. Dans notre cas, nous avons retrouvé exactement le même nombre d'enregistrements (48 livres) que dans BIBLIOPDB, confirmant qu'aucune perte de données n'est survenue pendant l'opération.

3.10 Étape 10 — Création d'un utilisateur applicatif

Afin de tester le clone dans des conditions proches d'un usage réel, nous avons créé un utilisateur local dédié à cette PDB, distinct des comptes administratifs.

```
SQL> CREATE USER test_clone IDENTIFIED BY "TestClone_2026"  
      DEFAULT TABLESPACE users  
      QUOTA UNLIMITED ON users;
```

User created.

```
SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW TO test_clone;
```

Grant succeeded.

Extrait de console 11 – Création d'un utilisateur local de test dans la PDB clonée

3.11 Étape 11 — Création d'une table et insertion de données

Connectés sous ce nouvel utilisateur, nous avons créé une table de test simple et inséré quelques enregistrements, afin de vérifier que la PDB clonée se comporte exactement comme une base de données autonome, capable d'évoluer indépendamment de sa source.

```
$ sqlplus test_clone/TestClone_2026@localhost:1521/bibliopdb_clone
```

```
SQL> CREATE TABLE test_verification (  
      id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
      commentaire VARCHAR2(100),  
      date_test DATE DEFAULT SYSDATE  
  );
```

Table created.

```
SQL> INSERT INTO test_verification (commentaire)
      VALUES ('Premier test apres clonage de la PDB');
```

```
1 row created.
```

```
SQL> COMMIT;
Commit complete.
```

Extrait de console 12 – Création de table et insertion de données dans la PDB clonée

[Emplacement réservé pour capture d'écran — connexion applicative à BIBLIOPDB_CLONE et insertion de données réussie]

Figure 5 – Test applicatif sur la PDB clonée, confirmant son fonctionnement autonome

3.12 Étape 12 — Test de non-impact sur la source

Pour confirmer l'indépendance totale entre la PDB source et son clone, nous avons vérifié, en revenant sur BIBLIOPDB, que la table TEST_VERIFICATION créée dans le clone n'existait pas dans la source. Ce test, simple en apparence, est important car il valide concrètement la séparation logique entre les deux bases après l'opération de clonage.

```
SQL> ALTER SESSION SET CONTAINER = bibliopdb;
Session altered.
```

```
SQL> SELECT table_name FROM user_tables WHERE table_name =
      'TEST_VERIFICATION';
```

```
no rows selected
```

Extrait de console 13 – Confirmation de l'absence d'impact du clone sur la PDB source

3.13 Étape 13 — Suppression éventuelle de la PDB clonée

Une fois les tests terminés, il est possible de supprimer la PDB clonée si elle n'a plus d'utilité, afin de libérer l'espace disque qu'elle occupe. Cette étape n'est pas systématique : dans un contexte réel, le clone peut au contraire être conservé comme environnement de test permanent. Nous l'avons réalisée ici uniquement à titre de démonstration, après avoir conservé une trace de tous les résultats précédents.

```
SQL> ALTER PLUGGABLE DATABASE bibliopdb_clone CLOSE IMMEDIATE;
Pluggable database altered.
```

```
SQL> DROP PLUGGABLE DATABASE bibliopdb_clone INCLUDING DATAFILES;
```

Pluggable database dropped.

Extrait de console 14 – Fermeture puis suppression complète de la PDB clonée et de ses fichiers

La clause INCLUDING DATAFILES est essentielle : sans elle, Oracle supprime uniquement les métadonnées de la PDB du dictionnaire de données, mais laisse les fichiers physiques sur le disque, ce qui peut conduire à un gaspillage d'espace disque si l'opération est répétée plusieurs fois au cours de tests successifs — une erreur que nous avons d'ailleurs commise lors d'un essai précédent, avant de relire attentivement la documentation Oracle sur ce point.

Chapitre 4 — Analyse des résultats

Au-delà de la simple réussite technique de la manipulation, il nous a semblé important de prendre du recul sur les résultats obtenus, en particulier sur les aspects de performance, de consommation de ressources et d'impact administratif du clonage de PDB. Cette analyse reste bien entendu limitée par l'échelle modeste de notre environnement de test, mais elle permet de dégager des tendances cohérentes avec ce que décrit la documentation Oracle pour des environnements de production de plus grande taille.

4.1 Temps de clonage

Le clonage de notre PDB BIBLIOPDB, contenant un volume de données très modeste (quelques dizaines de mégaoctets répartis sur trois tables principales), s'est achevé en un peu moins de deux minutes, comme indiqué au chapitre précédent. Ce temps est presque exclusivement consommé par la copie physique des fichiers de données sur le disque ; la partie purement « logique » de l'opération (mise à jour du dictionnaire de données, création des entrées de contrôle) ne représente qu'une fraction de seconde.

Cette observation nous amène à une conclusion assez intuitive mais importante : le temps de clonage est, pour l'essentiel, proportionnel à la taille des fichiers de données de la PDB source, et non à la complexité de son schéma ou au nombre d'objets qu'elle contient. Un schéma comportant des centaines de tables mais peu de données serait cloné presque aussi rapidement qu'un schéma simple, tandis qu'une base volumineuse, même avec peu de tables, prendrait nettement plus de temps.

4.2 Comparaison avec d'autres méthodes de duplication

Méthode de duplication	Temps estimé (pour notre volume de données)	Complexité de mise en œuvre
Clonage de PDB (CREATE PLUGGABLE DATABASE ... FROM)	≈ 1 min 40 s	Faible — une seule commande SQL
Export/Import Data Pump (expdp / impdp)	≈ 8 à 10 minutes (export + import)	Moyenne — deux opérations distinctes, gestion des fichiers dump
Sauvegarde RMAN puis restauration complète	≈ 12 à 15 minutes	Élevée — nécessite une stratégie de sauvegarde déjà en place
Recréation manuelle du schéma + scripts d'insertion	Variable, potentiellement plusieurs heures	Élevée — dépend de la disponibilité et de la fiabilité des scripts

Tableau 6 – Comparaison du temps et de la complexité entre le clonage de PDB et d'autres méthodes de duplication

Ce tableau illustre bien l'avantage déterminant du clonage de PDB par rapport aux méthodes traditionnelles : non seulement il est plus rapide, mais il nécessite également une seule instruction SQL, contre plusieurs étapes manuelles pour les autres approches. Il convient toutefois de noter que ces écarts se réduiraient sur des volumes de données extrêmement importants, où le temps de copie physique des fichiers deviendrait le facteur dominant pour toutes les méthodes.

4.3 Espace disque

Le clonage d'une PDB duplique intégralement ses fichiers de données. Dans notre cas, la PDB source occupait environ 1,2 Go sur le disque (essentiellement les fichiers SYSTEM, SYSAUX et le tablespace applicatif), et la PDB clonée a occupé un espace quasiment identique une fois la copie achevée. Cette consommation supplémentaire, négligeable à notre échelle, deviendrait un facteur de décision important dans un contexte de production où les PDB peuvent atteindre plusieurs centaines de gigaoctets : chaque clonage représenterait alors un coût de stockage non négligeable, ce qui explique l'intérêt des mécanismes de type Snapshot Copy évoqués au chapitre 1, que nous n'avons pas pu tester faute d'environnement de stockage compatible.

4.4 Performances pendant et après le clonage

Nous avons observé, pendant la durée du clonage, une légère augmentation de l'activité disque sur la machine virtuelle (visible via la commande iostat), sans toutefois constater de ralentissement perceptible des requêtes exécutées en parallèle sur la PDB source. Ce constat, bien que réalisé sur un environnement de taille réduite, est cohérent avec le principe du clonage à chaud, qui est précisément conçu pour minimiser l'impact sur la base source pendant l'opération.

Après l'ouverture de la PDB clonée, les temps de réponse des requêtes que nous avons exécutées (SELECT simples, jointures sur les trois tables du schéma) se sont révélés identiques à ceux observés sur la PDB source, ce qui n'a rien de surprenant puisque le clone hérite exactement de la même structure et des mêmes statistiques d'optimiseur au moment de sa création.

4.5 Impact sur l'administration

Du point de vue administratif, la principale charge de travail supplémentaire introduite par le clonage concerne la gestion du cycle de vie de la nouvelle PDB : enregistrement de son état (SAVE STATE), vérification de son inscription auprès du listener, mise en place éventuelle d'une politique de sauvegarde dédiée si le clone doit être conservé durablement. Dans notre cas, où le clone n'était destiné qu'à des tests ponctuels, cette charge est restée minime, mais elle mériterait d'être anticipée dans un contexte où de nombreux clonages seraient réalisés régulièrement, par exemple dans une chaîne d'intégration continue.

4.6 Synthèse des avantages observés et des limites

4.6.1 Avantages observés

- rapidité d'exécution confirmée, même sur une machine virtuelle aux ressources limitées ;
- fidélité totale des données et de la structure entre la source et le clone ;
- absence d'interruption perceptible du service sur la PDB source pendant l'opération ;
- simplicité de mise en œuvre, une seule commande SQL suffisant à déclencher l'ensemble du processus.

4.6.2 Limites constatées

- consommation d'espace disque proportionnelle à la taille de la source, sans optimisation copy-on-write dans notre configuration ;
- nécessité de plusieurs étapes complémentaires (ouverture, SAVE STATE, vérification réseau) pour rendre le clone pleinement opérationnel, qui ne sont pas automatiquement enchaînées par la seule commande de clonage ;
- dépendance forte à la configuration préalable du CDB (mode ARCHIVELOG notamment), qui peut surprendre un administrateur découvrant cette fonctionnalité pour la première fois.

Chapitre 5 — Difficultés rencontrées

Comme c'est souvent le cas lors d'une première manipulation d'administration Oracle, notre travail n'a pas été exempt d'erreurs et de blocages. Nous avons choisi de consacrer un chapitre entier à ces difficultés, non pas pour masquer les imperfections de notre démarche, mais parce qu'elles nous semblent au contraire pédagogiquement utiles : comprendre pourquoi une commande échoue permet souvent de mieux saisir le fonctionnement interne du système que la simple lecture de la documentation.

5.1 PDB non ouverte au moment du clonage

Lors de notre première tentative, nous avons exécuté la commande de clonage alors que la PDB source venait d'être redémarrée et se trouvait encore à l'état MOUNTED, faute d'avoir vérifié son état au préalable.

```
SQL> CREATE PLUGGABLE DATABASE bibliopdb_clone FROM bibliopdb
      FILE_NAME_CONVERT = ('bibliopdb', 'bibliopdb_clone');

ORA-65011: Pluggable Database BIBLIOPDB does not exist.
```

Extrait de console 15 – Erreur ORA-65011 rencontrée lors d'une tentative de clonage à partir d'une PDB mal référencée

Ce message d'erreur, bien que sa formulation laisse penser que la PDB n'existe pas du tout, provient en réalité d'une mauvaise correspondance entre le nom de la PDB indiqué dans la commande et le nom réellement enregistré dans le dictionnaire de données du CDB (un problème de casse ou une faute de frappe minime suffit à provoquer cette erreur). Dans notre cas, le problème venait d'une PDB que nous avions renommée lors d'un essai précédent, sans mettre à jour le script utilisé pour le clonage.

Solution retenue : nous avons systématiquement vérifié le nom exact des PDB via `SELECT name FROM v$pdbs` avant de lancer toute commande de clonage, et avons pris l'habitude de copier-coller le nom directement depuis le résultat de cette requête plutôt que de le saisir de mémoire.

5.2 Mode NOARCHIVELOG incompatible avec le clonage à chaud

Comme mentionné au chapitre 2, notre première installation du CDB était restée en mode NOARCHIVELOG par défaut. Notre première tentative de clonage à chaud s'est donc heurtée à l'erreur suivante :

```
SQL> CREATE PLUGGABLE DATABASE bibliopdb_clone FROM bibliopdb
      FILE_NAME_CONVERT = ('bibliopdb', 'bibliopdb_clone');
```

ORA-65016: FILE_NAME_CONVERT must be specified to rename the PDB datafiles.

Extrait de console 16 – Erreur ORA-65016 liée à une clause FILE_NAME_CONVERT incomplète

Cette erreur, dans notre cas précis, provenait en réalité d'un format incorrect de la clause FILE_NAME_CONVERT : nous avons indiqué des fragments de chemin trop courts, qui ne correspondaient pas exactement aux chemins réels des fichiers de la PDB source, ce qui empêchait Oracle de déterminer sans ambiguïté comment renommer chaque fichier.

Solution retenue : nous avons d'abord interrogé la vue CDB_DATA_FILES pour obtenir les chemins complets et exacts des fichiers de la PDB source, avant de reconstruire la clause FILE_NAME_CONVERT avec ces chemins complets plutôt qu'avec des fragments partiels.

```
SQL> SELECT file_name FROM cdb_data_files
       WHERE con_id = (SELECT con_id FROM v$pdb WHERE name = 'BIBLIOPDB');

FILE_NAME
-----
/u01/app/oracle/oradata/CDB19C/bibliopdb/system01.dbf
/u01/app/oracle/oradata/CDB19C/bibliopdb/sysaux01.dbf
/u01/app/oracle/oradata/CDB19C/bibliopdb/biblio_data01.dbf
```

Extrait de console 17 – Récupération des chemins exacts des fichiers de données avant correction de la commande

5.3 Conflit de noms de fichiers (ORA-65114)

Une fois la clause FILE_NAME_CONVERT corrigée, une nouvelle erreur est apparue lors d'un essai où nous avons, par erreur, fait pointer le répertoire cible vers le même répertoire que la source :

ORA-65114: space usage in the root is more than threshold value
ORA-65114: file name conflict with another PDB in the same container.

Extrait de console 18 – Erreur ORA-65114 liée à un conflit de chemin entre la source et la cible

Solution retenue : nous avons systématiquement utilisé un répertoire distinct pour chaque nouvelle PDB clonée, en respectant la convention de nommage /u01/app/oracle/oradata/CDB19C/<nom_pdb>/, et avons vérifié, avant chaque nouvelle tentative, que le répertoire cible n'existait pas encore ou était vide, à l'aide d'une simple commande ls depuis le terminal du système d'exploitation.

5.4 Fichier manquant lors de l'ouverture du clone (ORA-01109 / fichiers introuvables)

Lors d'un essai ultérieur, après avoir interrompu manuellement une opération de clonage jugée trop longue (avant de comprendre que ce temps d'attente était normal), nous nous sommes

retrouvés avec une PDB partiellement créée, dont certains fichiers de données étaient absents ou incomplets. La tentative d'ouverture de cette PDB a alors échoué :

```
SQL> ALTER PLUGGABLE DATABASE bibliopdb_clone OPEN;

ORA-01109: database not open
ORA-01157: cannot identify/lock data file 6 - see DBWR trace file
ORA-01110: data file 6:
'/u01/app/oracle/oradata/CDB19C/bibliopdb_clone/biblio_data01.dbf'
```

Extrait de console 19 – Erreurs ORA-01109 / ORA-01157 / ORA-01110 consécutives à une interruption prématurée du clonage

Cette situation nous a beaucoup appris sur l'importance de ne jamais interrompre une commande de clonage en cours d'exécution, même si elle paraît anormalement longue : la copie des fichiers physiques n'étant pas une opération transactionnelle réversible de la même façon qu'une instruction DML classique, une interruption laisse le système dans un état incohérent qu'il est difficile de corriger autrement qu'en repartant de zéro.

Solution retenue : nous avons supprimé la PDB partiellement créée avec la clause INCLUDING DATAFILES, vérifié manuellement l'absence de fichiers résiduels dans le répertoire correspondant, puis relancé l'intégralité de la commande de clonage sans l'interrompre cette fois, en patientant jusqu'à son achèvement complet.

5.5 Listener n'enregistrant pas le nouveau service

Après l'ouverture réussie de la PDB clonée, nous avons tenté de nous y connecter depuis SQL Developer, installé sur le poste hôte, en utilisant le nom de service BIBLIOPDB_CLONE. La connexion a échoué avec un message indiquant que le service demandé n'était pas connu du listener.

```
$ lsnrctl status
...
Services Summary...
Service "bibliopdb" has 1 instance(s).
Service "cdb19c" has 1 instance(s).
(Le service "bibliopdb_clone" n'apparaît pas encore dans la liste)
```

Extrait de console 20 – Absence temporaire du service de la PDB clonée dans le listener

Cette étape nous a permis de comprendre que l'enregistrement dynamique d'un service PDB auprès du listener n'est pas instantané : il dépend du paramètre service_names et du déclenchement périodique de l'enregistrement automatique par l'instance, qui peut prendre jusqu'à une à deux minutes. Plutôt que d'attendre, et pour disposer d'une solution plus fiable, nous avons forcé l'enregistrement immédiat avec la commande suivante :

```
SQL> ALTER SYSTEM REGISTER;
```

```
System altered.
```

Après cette commande, une nouvelle vérification via `lsnrctl status` a confirmé la présence du service `bibliopdb_clone`, et la connexion depuis SQL Developer a alors fonctionné normalement.

5.6 Problème de droits sur le répertoire cible

Un dernier incident, moins lié à Oracle lui-même qu'à la configuration système de la machine virtuelle, est survenu lorsque nous avons tenté de cloner une PDB vers un répertoire que nous avons créé manuellement en tant qu'utilisateur `root`, par habitude, sans modifier son propriétaire.

```
ORA-19505: failed to identify file  
"/u01/app/oracle/oradata/CDB19C/bibliopdb_clone/system01.dbf"  
ORA-27037: unable to obtain file status  
Linux Error: 13: Permission denied
```

Extrait de console 21 – Erreur de permissions Linux empêchant l'écriture des fichiers du clone

Solution retenue : nous avons modifié le propriétaire du répertoire concerné avec la commande `chown oracle:oinstall`, conformément aux droits attendus par les processus Oracle, qui s'exécutent sous le compte système `oracle` et non sous `root`. Cet incident, bien qu'extérieur à la logique du clonage proprement dit, nous a rappelé l'importance de respecter scrupuleusement la convention de propriété des fichiers recommandée par Oracle dès la phase d'installation.

5.7 Synthèse

Erreur / code	Cause identifiée	Solution appliquée
ORA-65011	Nom de PDB incorrect ou inexistant dans le contexte courant	Vérification du nom exact via <code>V\$PDBS</code> avant chaque commande
ORA-65016	Clause <code>FILE_NAME_CONVERT</code> incomplète ou mal formée	Récupération des chemins exacts via <code>CDB_DATA_FILES</code>
ORA-65114	Conflit de chemin entre fichiers source et cible	Répertoire cible distinct et vérifié avant clonage
ORA-01109 / ORA-01157 / ORA-01110	Interruption prématurée de la commande de clonage	Suppression du clone incomplet et nouvelle tentative sans interruption
Service absent du listener	Délai d'enregistrement dynamique du service PDB	Commande <code>ALTER SYSTEM REGISTER</code> pour forcer l'enregistrement
ORA-19505 / ORA-27037 (Permission denied)	Propriétaire incorrect du répertoire cible (<code>root</code> au lieu de <code>oracle</code>)	Correction des droits avec <code>chown oracle:oinstall</code>

Tableau 7 – Synthèse des difficultés rencontrées au cours de la manipulation et des solutions apportées

Conclusion générale

Ce travail nous a permis de prendre la mesure concrète de ce que représente l'architecture Multitenant d'Oracle 19c, au-delà de sa simple présentation théorique en cours. Manipuler réellement les commandes de création et de clonage de PDB, se confronter aux erreurs ORA-65011, ORA-65016 ou encore aux problèmes de permissions Linux, nous a fait comprendre des détails que la lecture seule de la documentation officielle ne permet pas toujours de saisir pleinement.

Sur le plan technique, le clonage local à chaud d'une PDB s'est révélé être une opération à la fois simple dans son principe — une seule commande SQL suffit — et exigeante dans sa mise en œuvre rigoureuse, notamment en ce qui concerne la configuration préalable du conteneur (mode ARCHIVELOG), la précision des chemins de fichiers, et la gestion des droits système. Nous retenons de cette expérience qu'une bonne partie des difficultés rencontrées ne provenait pas de la commande de clonage elle-même, mais de la préparation insuffisante de l'environnement qui l'entoure, ce qui est sans doute une leçon transposable à de nombreuses autres opérations d'administration de bases de données.

D'un point de vue plus personnel, ce module nous a confortés dans l'idée que l'administration de bases de données ne se réduit pas à l'apprentissage de syntaxes SQL, mais nécessite une compréhension fine de l'architecture sous-jacente du système, ainsi qu'une bonne dose de méthode et de patience face aux messages d'erreur, qui sont souvent plus instructifs qu'ils ne le paraissent au premier abord. Nous avons par ailleurs pu réinvestir, dans ce travail, certains réflexes de rigueur déjà développés lors de la mise en place de notre projet Bibliothèque API pour le module d'Architecture Orientée Services, notamment l'habitude de vérifier systématiquement l'état d'un système avant d'y appliquer une modification.

Ce rapport ouvre naturellement la voie à plusieurs prolongements possibles. Le premier concerne Oracle 21c, qui généralise certains mécanismes du Multitenant déjà évoqués dans ce document, en particulier l'abandon progressif du mode non-CDB et le renforcement des fonctionnalités de clonage rapide. Le second concerne l'environnement Oracle Cloud Infrastructure, où l'essentiel des opérations que nous avons réalisées manuellement en ligne de commande se trouve aujourd'hui automatisé derrière une interface graphique ou une API REST, à travers des services comme Oracle Autonomous Database. Comprendre, comme nous avons tenté de le faire ici, les mécanismes internes qui se cachent derrière ces automatismes nous semble être un atout pour aborder ces environnements cloud avec un regard plus critique et plus averti, plutôt que de se contenter d'utiliser ces outils comme de simples boîtes noires.

Annexes

Cette section regroupe, à titre de référence, l'ensemble des commandes SQL utilisées au cours de la manipulation décrite au chapitre 3, ainsi que les principaux paramètres de configuration de l'environnement.

Annexe A — Script complet de clonage

```
-- Connexion SYSDBA
sqlplus / as sysdba

-- Vérification des PDB existantes
SELECT con_id, name, open_mode FROM v$pdb;

-- Vérification du contenu de la PDB source
ALTER SESSION SET CONTAINER = bibliopdb;
SELECT table_name FROM user_tables;
ALTER SESSION SET CONTAINER = CDB$ROOT;

-- Récupération des chemins exacts des fichiers de la source
SELECT file_name FROM cdb_data_files
WHERE con_id = (SELECT con_id FROM v$pdb WHERE name = 'BIBLIOPDB');

-- Commande de clonage local à chaud
CREATE PLUGGABLE DATABASE bibliopdb_clone
  FROM bibliopdb
  FILE_NAME_CONVERT = ('/u01/app/oracle/oradata/CDB19C/bibliopdb/',
                      '/u01/app/oracle/oradata/CDB19C/bibliopdb_clone/');

-- Ouverture du clone
ALTER PLUGGABLE DATABASE bibliopdb_clone OPEN;

-- Persistance de l'état à travers les redémarrages
ALTER PLUGGABLE DATABASE bibliopdb_clone SAVE STATE;

-- Vérification finale
SELECT con_id, name, open_mode FROM v$pdb;

-- Forcer l'enregistrement du service auprès du listener si nécessaire
ALTER SYSTEM REGISTER;

-- Suppression complète du clone (si besoin)
ALTER PLUGGABLE DATABASE bibliopdb_clone CLOSE IMMEDIATE;
DROP PLUGGABLE DATABASE bibliopdb_clone INCLUDING DATAFILES;
```

Annexe A – Script SQL complet de la procédure de clonage local à chaud utilisée dans ce rapport

Annexe B — Création de l'utilisateur de test et table de vérification

```
ALTER SESSION SET CONTAINER = bibliopdb_clone;

CREATE USER test_clone IDENTIFIED BY "TestClone_2026"
  DEFAULT TABLESPACE users
  QUOTA UNLIMITED ON users;

GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW TO test_clone;

CREATE TABLE test_verification (
  id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  commentaire VARCHAR2(100),
  date_test DATE DEFAULT SYSDATE
);

INSERT INTO test_verification (commentaire)
VALUES ('Premier test apres clonage de la PDB');

COMMIT;
```

Annexe B – Script de création de l'utilisateur applicatif et de la table de vérification

Annexe C — Paramètres de configuration du CDB

```
-- Vérification du mode d'archivage
SELECT log_mode FROM v$database;

-- Passage en mode ARCHIVELOG (si nécessaire, depuis l'état MOUNT)
SHUTDOWN IMMEDIATE;
STARTUP MOUNT;
ALTER DATABASE ARCHIVELOG;
ALTER DATABASE OPEN;

-- Vérification du listener
lsnrctl status

-- Correction des droits sur un répertoire cible (depuis le terminal, en root)
chown -R oracle:oinstall /u01/app/oracle/oradata/CDB19C/bibliopdb_clone
```

Annexe C – Commandes de configuration du conteneur et de correction des droits système